

polyloop - ewem green hackathon

Project Blueprint: E-Waste Plastic Identification & Reuse System

Focus: Circular Economy Engineering, Automated Polymer Classification, and Closed-Loop Upcycling

Target Environment: Agent IDEs (Claude Code, Antigravity, Codex)

Team Allocation: 4 Engineers

1. Executive Summary & Strategic Value

Electronic waste plastics (predominantly **ABS, HIPS, PC, and PC-ABS blends**) account for nearly **20% to 30% of total e-waste volume**, yet over 80% ends up landfilled or incinerated. The core technical bottleneck in recycling e-waste plastic is twofold:

1. **Polymer Incompatibility:** Mingling polymers (e.g., cross-contaminating ABS with HIPS) ruins tensile strength during melt-processing.
2. **Hazardous Additives:** The presence of toxic **Brominated Flame Retardants (BFRs)** (regulated under RoHS/WEEE directives) prohibits reuse in consumer goods or 3D printing filaments.

What the System Delivers

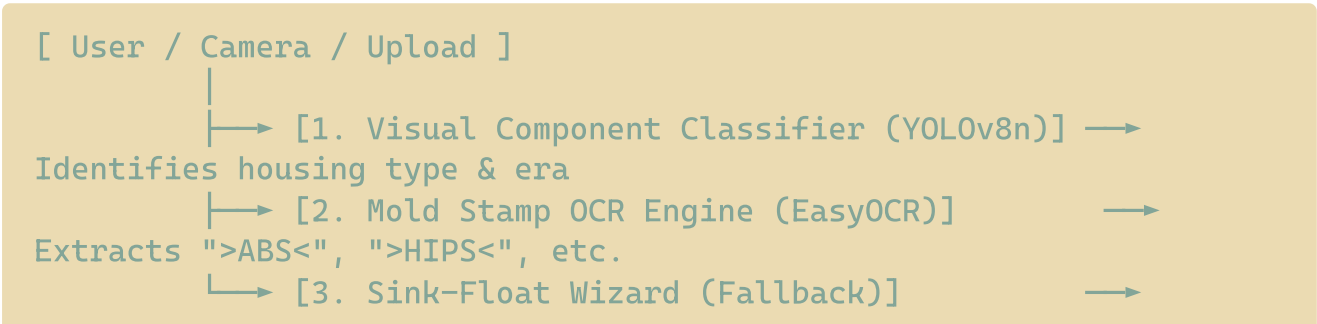
This system provides an automated pipeline for sorting, auditing, and upcycling discarded casings:

- **Hybrid Identification:** Visual identification of device components + OCR recognition of engraved mold stamps (**>ABS<**, **>HIPS<**, **>PC+ABS<**) + a guided physical diagnostic fallback (density / sink-float testing).
- **BFR Contamination Risk Engine:** Predicts flame-retardant risk based on appliance category and era of manufacture.
- **Reuse Pathway & Upcycling Engine:** Calculates optimal downstream processing (e.g., melt-extrusion temperatures for DIY 1.75 mm 3D printer filament, injection molding parameters, or composite construction aggregate).
- **Digital Material Passport:** Generates an automated, downloadable batch audit PDF detailing polymer purity, circularity index, and carbon avoidance metrics.

2. 100% Free Technology Stack

Layer	Technology	Free Tier / Open-Source Justification
Vision & OCR	Ultralytics YOLOv8 / YOLOv11n + EasyOCR	Open-source (AGPL/Apache). Runs locally on CPU/GPU without API subscriptions.
Image Processing	OpenCV (<code>opencv-python-headless</code>)	High-performance preprocessing (adaptive thresholding, morphology filters for engraved text).
Backend API	FastAPI + Pydantic v2	Python-native, automated OpenAPI docs, seamless integration with Python ML/data libraries.
Local Database	SQLite + SQLAlchemy / SQLModel	Embedded serverless database, zero setup, easy to version-control schema migrations.
Frontend UI	React (Vite) + Tailwind CSS + Lucide React	Fast build times, standard styling primitives, easy for Agent IDEs to scaffold clean dashboards.
Data Viz	Recharts (React)	Component-based SVG charts for polymer composition, tensile retention, and temperature curves.
Passport Generator	ReportLab (Python)	Open-source PDF generator to produce verifiable "E-Waste Plastic Material Passports".
CI/CD & Hosting	GitHub Actions + Vercel / Localhost	Free cloud CI runs; zero-cost frontend deployment.

3. System Architecture & Information Flow



Stepwise water/alcohol density test

[FastAPI Core Processing Service]

[BFR & RoHS Risk Scorer]

Thermal Matchmaker]

- Historical era heuristic filament suitability
- CRT/Printer/Monitor risk flags (210–240°C) & bed temps
- Regulatory compliance check circularity & CO2 offset

[Upcycling &

- 3D Printing
- Extrusion temp
- Material

[Interactive Web Dashboard + Downloadable Material Passport (PDF)]

4. Work Division for 4 Team Members

| TEAM LEAD |

| Architecture & Repo Governance |

Member 1 Member 2 Member 3

Vision & OCR Core Backend Frontend UI

Pipeline & Polymer Engine & HUD Lead

| | |

Member 4

Circularity, Thermal

Calculators & QA

Member 1: Vision, OCR & Identification Pipeline Lead

- **Primary Scope:** Detection models, image preprocessing, and optical character recognition for resin markings.
- **Core Responsibilities:**
 - Train/configure lightweight YOLOv8n to identify common e-waste plastic parts (monitor backings, keyboard casings, PC chassis, printer panels).
 - Build an image enhancement pipeline with OpenCV to detect stamped injection marks (e.g., ISO 11469 symbols like `>ABS<`, `>PS-HI<`, `>PC+ABS<`).
 - Implement EasyOCR wrapper that extracts stamped text from cropped regions of interest (ROI).
 - Provide an inference utility script returning normalized JSON payloads:

```
{  
  polymer_detected: "ABS", confidence: 0.94, stamp_found: true  
}
```

.

Member 2: Backend Architecture & Polymer Knowledge Engine

- **Primary Scope:** API architecture, database persistence, and deterministic polymer classification rules.
- **Core Responsibilities:**
 - Build FastAPI application structure with routers, schemas, CORS middleware, and error handling.
 - Construct the **Polymer Knowledge Base (PKB)**: Density, melting point, glass transition temperature (T_g), solvent reaction (acetone test response), and mechanical properties.
 - Implement the **BFR (Brominated Flame Retardants) Risk Engine**: Calculate hazard probabilities based on manufacturing year, device category, and color (e.g., black/grey CRT casings = high risk).
 - Design SQLite schema for storing sample scan logs, batch runs, and audit histories.

Member 3: Frontend UI, Webcam HUD & Diagnostic Wizard

- **Primary Scope:** Complete user interface, camera feed integration, and interactive test workflows.
- **Core Responsibilities:**
 - Build modern responsive dashboard (React + Tailwind CSS) with a clean dark-mode industrial theme.

- Create webcam capture interface with live bounding-box targeting overlay (HTML5 Canvas).
- Implement the **Interactive Sink-Float & Flame Test Wizard**: A dynamic step-by-step questionnaire (e.g., "Does it float in pure water? [Yes/No] -> Does it dissolve in Acetone? [Yes/No]") that deduces polymer type when no stamps are visible.
- Build real-time display cards for thermal specs, safety warnings, and identified polymer properties.

Member 4: Upcycling Matchmaker, LCA Metrics & QA Lead

- **Primary Scope**: Downstream processing calculations, material passport generation, CI/CD, and repository testing.
- **Core Responsibilities**:
 - Build the **Thermal & Extrusion Parameter Calculator**: Given an identified polymer, outputs precise 3D filament extrusion specs (nozzle temp, bed temp, cooling rate, shrinkage compensation).
 - Develop the **Lifecycle Assessment (LCA) Engine**: Calculates virgin plastic substitution credits, landfill mass diverted, and net CO_2 equivalent emissions prevented.
 - Implement the PDF export service using ReportLab to generate an industrial **"E-Waste Plastic Batch Passport"**.
 - Set up GitHub Actions CI/CD pipeline (Ruff linter, Pytest test suites) and prepare synthetic seed datasets for demo day.

5. GitHub Branching Strategy & Workflow

Repository Rules

- **Protected Branches**:
 - `main`: Production release only. Code must be stable, demo-ready, and tagged.
 - `develop`: Integration staging. All features merge here via Pull Request.
- **Workflow**: No direct pushes to `main` or `develop`. Every PR requires at least **one peer review approval** and green CI checks.

Plaintext

```
main (v1.0-release)
└─ develop (Integration Branch)
    ├── feat/m1-yolo-casing-classifier
    ├── feat/m1-ocr-stamp-extractor
    ├── feat/m2-fastapi-core-architecture
    ├── feat/m2-polymer-rules-engine
    ├── feat/m3-react-dashboard-shell
    ├── feat/m3-sink-float-wizard-ui
    ├── feat/m4-extrusion-calc-engine
    └─ feat/m4-pdf-passport-generator
```

Complete Branch Schedule by Member

Member 1 (Vision & OCR)

- `feat/m1-yolo-casing-classifier`: Sets up YOLOv8 inference script to classify electronic casing categories.
- `feat/m1-opencv-preprocessing`: Adaptive contrast adjustment, thresholding, and edge enhancement for stamped plastic symbols.
- `feat/m1-ocr-stamp-extractor`: EasyOCR integration to parse ISO standard resin text (`>ABS<` , `>HIPS<`).
- `feat/m1-inference-service-wrap`: Bundles models into a single clean Python inference class callable by FastAPI.

Member 2 (Backend & Domain Engine)

- `feat/m2-fastapi-core-architecture`: Scaffolds FastAPI skeleton, router definitions, logging, and CORS configuration.
- `feat/m2-polymer-rules-engine`: Implementation of the polymer property database and classification logic.
- `feat/m2-bfr-risk-scoring`: Risk assessment algorithm for flame retardants based on age and equipment type.
- `feat/m2-sqlite-database-crud`: SQLite models and repositories for saving scan batches and test records.

Member 3 (Frontend & Diagnostic UI)

- `feat/m3-react-dashboard-shell`: Vite + React + Tailwind project initialization, layout, navbar, and status cards.

- `feat/m3-webcam-canvas-hud`: Camera streaming component with snapshot capture and upload dropzone.
- `feat/m3-sink-float-wizard-ui`: Multi-step interactive decision wizard for manual density/sink-float tests.
- `feat/m3-metrics-and-graphs`: Recharts visualizations for batch statistics, thermal profiles, and polymer distribution.

Member 4 (LCA, PDF & DevOps)

- `feat/m4-extrusion-calc-engine`: Computational logic for 3D printer filament extrusion settings and mechanical yields.
- `feat/m4-lca-carbon-calculator`: Environmental impact calculator ($kg\ CO_2e$ avoided vs. virgin plastic production).
- `feat/m4-pdf-passport-generator`: ReportLab script generating structured PDF technical datasheets.
- `feat/m4-ci-pipeline-and-mockdata`: GitHub Actions workflow config and mock sample images/data for automated tests.

6. Sprints & Milestone Execution Plan

Sprint 1: Scaffolding & Foundational Logic (Days 1–3)

- **Goal:** Core environments established, dummy endpoints communicating, UI skeleton operational.
- **M1:** Assemble test dataset of 20 e-waste plastic casing images (with and without stamps).
- **M2:** Create FastAPI project with basic endpoints returning mock polymer JSON data.
- **M3:** Build clean React landing dashboard connected to mock FastAPI endpoints.
- **M4:** Write mathematical formulas for virgin plastic displacement and carbon savings; configure GitHub repo settings and branch protection.

Sprint 2: Core Feature Implementation (Days 4–7)

- **Goal:** Functional identification models, real database records, and dynamic diagnostic UI.
- **M1:** Finish EasyOCR stamp extraction and integrate confidence scoring.

- **M2:** Integrate BFR risk calculation and wire SQLite persistence for batch sessions.
- **M3:** Build the Interactive Sink-Float Guided Flow and Camera HUD overlay.
- **M4:** Implement thermal parameter calculator and PDF report layout generator.

Sprint 3: Integration, Polish & Evaluation Demo (Days 8–10)

- **Goal:** End-to-end integration into `develop`, edge-case testing, and final merge into `main`.
- **Team:**
 1. Connect frontend camera upload directly to M1's OCR pipeline via M2's API.
 2. Test fallback routing: if OCR fails to read a stamp, automatically route user to M3's Sink-Float wizard.
 3. Validate PDF generation: Ensure clicking "Download Material Passport" provides a complete technical datasheet.
 4. Final review, squash merge `develop` into `main`, and freeze demo environment.

7. Pull Request (PR) Quality Gate Checklist

Before any PR is approved into `develop`:

- ☐ **Branch Source:** PR is created against `develop` (not `main`).
- ☐ **Execution Test:** Code runs locally without errors (`python -m pytest` passes, `npm run build` succeeds).
- ☐ **Zero API Costs:** No commercial external API keys (OpenAI, AWS, etc.) required; everything runs on local/free libraries.
- ☐ **Clean Code:** Python code formatted via Ruff/Black; no dead code or unhandled Promise rejections in JavaScript.
- ☐ **Demo Resilience:** All endpoints handle unexpected/corrupt image inputs gracefully without server panics.